

20. page 61 , Ans page 196

Program 6 Stack structure(made using singly linked list):

```
class Stack {
    private Singly head;
    Stack() {
        this.head = null;
    }
    boolean isEmpty() {
        return this.head == null;
    }
    void push(int x) {
        this.head = new Singly(x, this.head);
    }
    int top() {
        if (this.head == null)
            throw new NoSuchElementException();
        return this.head.element;
    }
    int pop() {
        if (this.head == null)
            throw new NoSuchElementException();
        int e = this.head.element;
        this.head = this.head.next;
        return e;
    }
}
```

```
static String listToString(Singly s) {
    StringBuffer sb = new StringBuffer("[");
    while (s != null) {
        sb.append(s.element);
        if (s.next != null) sb.append(" -> ");
        s = s.next;
    }
    return sb.append("]").toString();
}
```

Write a public dynamic method String toString() for the Stack class, which returns the contents of a stack as a string such as "[1, 2, 3]" where 1 is the top of the stack. We can draw inspiration from the listToString method given above.

幫 Stack class 寫一個 method toString(), "[1,2,3]" 1 是 stack 的 top

可以從 listToString 獲得啟發

- Singly 是單向 linked list

```
String toString(){
    StringBuffer sb = new StringBuffer("[");
    while (s != null) {
        sb.append(s.element);
        if (s.next != null) sb.append(",");
        s = s.next;
    }
    return sb.append("]").toString();
}
```

24. page 70 , Ans page 196

Program 10 The Josephus Problem:

```
// constructs the circular list 1,2,...,n and
returns element 1
```

```
static Doubly circle(int n) {
    Doubly l1 = new Doubly(1);
    for (int i = n; i >= 2; i--) {
        l1.insertAfter(i);
        if (i == n) { l1.prev = l1.next; l1.next.next = l1; }
    }
    return l1;
}
static int josephus(int n, int p) {
    Doubly c = circle(n);
    while (c != c.next) { // as long as there are at
        least two players left
        for (int i = 1; i < p; i++)
            c = c.next;
        c.remove(); // we eliminate the p-th
        c = c.next;
    }
    return c.element;
}
```

Rewrite the josephus method using a singly linked cyclic list. Hint: in the inner loop, keep a pointer to the previous element, so that you can easily delete the p-th element at the end of the loop. 重寫 josephus method 使用 singly linked list, Hint: 在內部迴圈中，保持一個指向前一個元素的指針，以便在迴圈結束時可以輕鬆刪除第 p 個元素。

- doubly 是雙向 linked list

- Josephus problem:

- 有 n 個人 (編號從 1 到 n) 圍成一圈，從第 1 個人開始報數，每報到第 k 個人就將其淘汰，接著從下一個人繼續報數。如此重複進行，直到剩下一人為止。求最後剩下的那個人的編號。

```
static Singly circle(int n) {
    Singly first = new Singly(1, null);
    Singly last = first;
    for (int i = 2; i <= n; i++) {
        last.next = new Singly(i, first);
    }
    last.next = first;
    return first;
}
static int josephus(int n, int p) {
    Singly c = circle(n);
    while (c != c.next) { // as long as there are at
        least two players left
        for (int i = 1; i < p; i++)
            c = c.next;
        c.remove(); // we eliminate the p-th
        c = c.next;
    }
    return c.element;
}
```

28. page 75 , Ans page 198

```
class Bucket {
    String element;
    Bucket next;
    Bucket(String element, Bucket next) {
        this.element = element;
    }
}
```

```

        this.next = next;
    }
}
class HashTable {
    private Bucket[] buckets;
    final private static int M = 17;
    HashTable() {
        this.buckets = new Bucket[M];
    }

    private int hash(String s) {
        int h = 0;
        for (int i = 0; i < s.length(); i++)
            h = s.charAt(i) + 19 * h;
        return (h & 0x7fffffff) % M;
    }

    void add(String s) {
        int i = hash(s);
        this.buckets[i] = new Bucket(s,
this.buckets[i]);
    }

    void remove(String s){
        int i = hash(s);
        Bucket b = this.buckets[i];
        if (b == null) return; // nothing to remove

        if (b.element.equals(s)) {
            this.buckets[i] = b.next; // 刪除頭節點
            return;
        }

        for (; b.next != null; b = b.next){
            if (b.next.element.equals(s)){
                b.next = b.next.next;
                return;
            }
        }
    }
}

```

Add a void remove(String s) method to remove an element s from the hash table. What is the impact on the add method?

- 在 hash table 中加入一個 void remove(String s) method 來刪除元素 s，對 add method 有什麼影響？

37. page 84, Ans page 198

```

class BST {
    int value;
    BST left, right;
}

static int getMin(BST b) {
    while (b.left != null) b = b.left;
    return b.value;
}

static boolean contains(BST b, int x) {
    while (b != null) {
        if (x == b.value) return true;
        b = (x < b.value) ? b.left : b.right;
    }
    return false;
}

```

```

}

static int floor(BST b, int x){
    int maxLessThanX = null;
    while (b != null) {
        if (x == b.value) return x;
        if (x < b.value) b = b.left;
        else { maxLessThanX = b.value; b = b.right; }
    }

    if (maxLessThanX == null) throw new
NoSuchElementException();
    return maxLessThanX;
}

```

Binary search tree Write a method static int floor(BST b, int x) that returns the largest element of b less than or equal to x, if it exists, and throws an exception otherwise.

- 寫一個方法 static int floor(BST b, int x) 返回 b 中小於或等於 x 的最大元素，不到就拋 exception。

40. page 95, Ans page 199

```

class AVL {
    int value;
    AVL left, right;
    int height;
    AVL(AVL left, int value, AVL right) {
        this.left = left;
        this.value = value;
        this.right = right;
        this.height = 1 + Math.max(height(left),
height(right));
    }
    static int height(AVL a) {
        return (a == null) ? 0 : a.height;
    }

    static boolean contains(AVL a, int x)
    static int getMin(AVL b)
    static AVL removeMin(AVL b)
    static AVL remove(AVL b)

    private static AVL rotateRight(AVL t) {
        assert t != null && t.left != null;
        AVL l = t.left;
        t.left = l.right;
        l.right = t;
        t.height = 1 + Math.max(height(t.left),
height(t.right));
        l.height = 1 + Math.max(height(l.left),
height(l.right));
        return l;
    }
    private static AVL rotateLeft(AVL t) {
        assert t != null && t.right != null;
        AVL r = t.right;
        t.right = r.left;
        r.left = t;
        t.height = 1 + Math.max(height(t.left),
height(t.right));
        r.height = 1 + Math.max(height(r.left),
height(r.right));
        return r;
    }
    private static AVL balance(AVL t) {

```

```

assert t != null;
AVL l = t.left, r = t.right;
int hl = height(l), hr = height(r);
if (hl > hr + 1) {
    AVL ll = l.left, lr = l.right;
    if (height(ll) >= height(lr))
        return rotateRight(t);
    else {
        t.left = rotateLeft(t.left);
        return rotateRight(t);
    }
} else if (hr > hl + 1) {
    AVL rl = r.left, rr = r.right;
    if (height(rr) >= height(rl))
        return rotateLeft(t);
    else {
        t.right = rotateRight(t.right);
        return rotateLeft(t);
    }
} else {
    t.height = 1 + Math.max(hl, hr);
    return t;
}
}

class AVLSet {
    private AVL root;
    private int size;
    int size() {
        return this.size;
    }

    boolean isEmpty();
    boolean contains(int x);
    void add(int x){
        if (!AVL.contains(this.root, x)) {
            this.root = AVL.add(this.root, x);
            this.size++;
        }
    }
    void remove(int x){
        if (AVL.contains(this.root, x)) {
            this.root = AVL.remove(this.root, x);
            this.size--;
        }
    }
}

```

Add to the AVLSet class a private field size containing the number of elements in the set and an int size() method that returns its value. Modify the add and remove methods to update the value of this field. It will be necessary to correctly handle the case where the element added by add is already in the set and the case where the element removed by remove is not in the set.

- 在 AVLSet class 中加入一個 private field size，包含 set 中的元素數量，和一個 int size() 方法返回它的值。修改 add 和 remove 方法來更新 size 的值。需要正確處理 add 中添加的元素已經在 set 中的情況，以及 remove 中刪除的元素不在 set 中的情況。

```

class UnionFind {
    private int[] link;
    private int[] rank;
    private int numClasses;

    UnionFind(int n) {
        if (n < 0) throw new IllegalArgumentException();
        this.link = new int[n];
        for (int i = 0; i < n; i++) this.link[i] = i;
        this.rank = new int[n];
        this.numClasses = n; // initially, all elements
        // are in different classes
    }

    int find(int i) {
        if (i < 0 || i >= this.link.length)
            throw new ArrayIndexOutOfBoundsException(i);
        int p = this.link[i];
        if (p == i) return i;
        int r = this.find(p);
        this.link[i] = r;
        return r;
    }

    void union(int i, int j) {
        int ri = this.find(i);
        int rj = this.find(j);
        if (ri == rj) return;
        this.numClasses--; // we merge two classes
        if (this.rank[ri] < this.rank[rj])
            this.link[ri] = rj;
        else {
            this.link[rj] = ri;
            if (this.rank[ri] == this.rank[rj])
                this.rank[ri]++;
        }
    }

    boolean sameClass(int i, int j) {
        return this.find(i) == this.find(j);
    }

    int numClasses(){
        return this.numClasses;
    }
}

```

Add to the union-find structure a method int numClasses() giving the number of distinct classes. We will try to provide this value in constant time, maintaining the value as an additional field.

- 在 union-find 結構中加入一個方法 int numClasses()，給出不同類別的數量。我們會試著在常數時間內提供這個值，並將其作為額外的欄位維護。
- 剛 construct 出來的時候，numClasses = n
- 當 union 的時候，numClasses - -

61. page 127 , Ans 205

Another solution to realize the union-find structure is not to use arrays, but to represent

each element directly as an object containing two fields rank and link. If E denotes the type of the elements, we can define the following generic class:

```
class Elt<E> {
    private E value;
    private Elt<E> link;
    private int rank; // size
    ...

    Elt(E x){
        value = x;
        link = null;
        rank = 0;
    }

    Elt<E> find(){
        if (this.link == null) return this;
        Elt<E> r = this.link.find();
        this.link = r; // path compression
        return r;
    }

    void union(Elt<E> e){
        Elt<E> r1 = this.find();
        Elt<E> r2 = e.find();
        if (r1 == r2) return; // already in the same
class
        if (r1.rank < r2.rank) {
            r1.link = r2;
        } else {
            r2.link = r1;
            if (r1.rank == r2.rank) { r1.rank++; }
        }
    }
}
```

It is no longer necessary to maintain global information about the union-find structure, because each element contains all the necessary information. (Be careful, however, not to share a value of type Elt<E> between multiple partitions.) The interface of the Elt class is as follows:

The Elt(E x) constructor constructs a class containing a single element, of value x. We can choose the convention that a link pointer is null when it is a representative. Write this constructor as well as the find and union methods.

- 另一種實現 union-find 結構的方法是直接將每個元素表示為一個對象，該對象包含兩個欄位 rank 和 link。如果 E 表示元素的類型，我們可以定義以下通用類：

```
Elt(E x)
Elt<E> find()
void union(Elt<E> e
```

- 不再需要維護有關 union-find 結構的全局信息，因為每個元素都包含所有必要的信息。

78. page 143 , Ans page 207

Program 26 The total is good:

```
// computes all integers that can be constructed
with
// any subset of s
// a subset of s is a mask of {0,...,|s|-1}
// represented by an integer between 0 and 2^|s|-1
static Map<Integer, Map<Integer, String>>
computeAll(int[] s) {
    Map<Integer, Map<Integer, String>> res = new
HashMap<>();
    // we start with the singletons
    for (int i = 0; i < s.length; i++) {
        Map<Integer, String> m = new HashMap<>();
        m.put(s[i], "" + s[i]);
        res.put(1 << i, m);
    }
    int n = 1 << s.length;
    // then for any subset u
    for (int u = 3; u < n; u++) {
        if (res.containsKey(u)) // already done (u is a
singleton)
            continue;
        Map<Integer, String> m = new HashMap<>();
        res.put(u, m);

        for (int left = 1; left < u; left++)
            if ((left & u) == left) { // left is a subset of
u
                int right = u & ~left;
                m.putAll(res.get(left));
                for (int x: res.get(left).keySet()) {
                    String ex = res.get(left).get(x);
                    for (int y: res.get(right).keySet()) {
                        String ey = res.get(right).get(y);
                        m.put(x + y, "(" + ex + "+" + ey + ")");
                        m.put(x * y, "(" + ex + "*" + ey + ")");
                        if (x >= y) m.put(x - y,
 "(" + ex + "-" + ey + ")");
                        if (y != 0 && x % y == 0) m.put(x / y,
 "(" + ex + "/" + ey + ")");
                    }
                }
            }
        }
    }
    return res;
}
```

Is Program 26 still correct if the array s contains duplicates, i.e., if it is a multiset and not a set?

- 程式 26 仍然正確嗎？如果數組 s 包含重複項，即如果它是一個多重集合而不是一個集合？

The program remains correct. In fact, we represent a subset of the array s by a mask of the selected indices and not as a subset of elements of this array.

- 該程序仍然正確。事實上，我們通過所選索引的掩碼而不是該數組的元素子集來表示數組 s 的子集。

44. page 100, Ans page 201

```
class Trie {
    private boolean word;
```

```

private HashMap<Character, Trie> branches;
Trie() {
    this.word = false;
    this.branches = new HashMap<Character, Trie>();
}
boolean contains(String s) {
    Trie t = this;
    for (int i = 0; i < s.length(); i++) { //
invariant t != null
        t = t.branches.get(s.charAt(i));
        if (t == null) return false;
    }
    return t.word;
}
void add(String s) {
    Trie t = this;
    for (int i = 0; i < s.length(); i++) { //
invariant t != null
        char c = s.charAt(i);
        Map<Character, Trie> b = t.branches;
        t = b.get(c);
        if (t == null) {
            t = new Trie();
            b.put(c, t);
        }
    }
    t.word = true;
}

void remove(String s){
    if (!contains(s)) return; // nothing to remove
    Trie t = this;
    for (int i=0; i<s.length(); i++){
        char c = s.charAt(i);
        t = t.branches.get(c);
        if (t == null) return; // nothing to remove
    }
    t.word = false; // remove the word flag
}
}

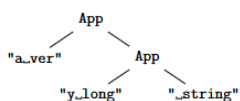
```

Add to the Trie class a method void remove(String s) which removes the occurrence of the string s, if it exists.

- 在 Trie class 中加入一個 void remove(String s)方法，刪除 s 的出現次數，如果存在的話。

51. page 108 , Ans page ??

rope tree



```

abstract class Rope {
    int length;
    Rope(int length) { this.length = length; }
    abstract char get(int i);
    abstract Rope sub(int begin, int end);
}
class Str extends Rope {
    String str;
    Str(String str) {
        super(str.length());
        this.str = str;
    }
}

```

```

}
char get(int i) {
    return this.str.charAt(i);
}

Rope sub(int begin, int end) {
    return new Str(this.str.substring(begin, end));
}

class App extends Rope {
    Rope left, right;
    App(Rope left, Rope right) {
        super(left.length + right.length);
        this.left = left;
        this.right = right;
    }
    char get(int i) {
        return (i < this.left.length) ?
            this.left.get(i) : this.right.get(i -
this.left.length);
    }
}

// mean a very long string
Rope r = new App(new Str("a ver"),
    new App( new Str("y long"),
        new Str(" string")));

```

Add a String toString() method that returns the Java string defined by a rope. We will take care to do this efficiently using a StringBuffer.

- 在 rope 中加入一個 String toString()方法，返回 rope 定義的 Java 字符串。

我們會注意使用 StringBuffer 來有效率地完成此工作。

```

String toString() {
    StringBuffer sb = new StringBuffer(this.length);
    for (int i = 0; i < this.length; i++)
        sb.append(this.get(i));
    return sb.toString();
}

```

Other

```

class Trie {
    private boolean word;
    private HashMap<Character, Trie> branches;

    public Trie() {
        word = false;
        branches = new HashMap<>();
    }

    public void insert(String word) {
        Trie node = this;
        for (char c : word.toCharArray()) {
            if (!node.branches.containsKey(c)) {
                node.branches.put(c, new Trie());
            }
            node = node.branches.get(c);
        }
        node.word = true;
    }
}

```

```
public boolean search(String word) {
    Trie node = this;
    for (char c : word.toCharArray()) {
        if (!node.branches.containsKey(c)) {
            return false;
        }
        node = node.branches.get(c);
    }
    return node.word;
}

public boolean startsWith(String prefix) {
    Trie node = this;
    for (char c : prefix.toCharArray()) {
        if (!node.branches.containsKey(c)) {
            return false;
        }
        node = node.branches.get(c);
    }
    return true;
}
}
```